

# Hospital Management System

## 1. Project Overview

The Hospital Management System is a full-stack web application developed to manage hospital operations related to patients, doctors, and appointments.

The system provides hospital staff and administrators with a centralized platform to create, view, update, search, filter, and manage patient and doctor records, as well as schedule and manage appointments.

The application follows a client-server architecture with a React frontend, Spring Boot REST backend, and MySQL database.

## 2. Project Objective

The primary objective of this project is to develop a simple, reliable, and user-friendly hospital management application that supports:

- Patient management
- Doctor management
- Appointment management
- Appointment conflict detection
- Appointment cancellation
- Patient appointment history
- Doctor appointment schedule
- Dashboard statistics
- Data validation
- Exception handling
- Persistent storage using MySQL

## 3. Users

The system is designed for:

## Hospital Staff

- Manage patient records
- View doctor information
- Schedule appointments
- Update appointments
- Cancel appointments
- View patient appointment history
- View doctor appointment schedules

## Administrator

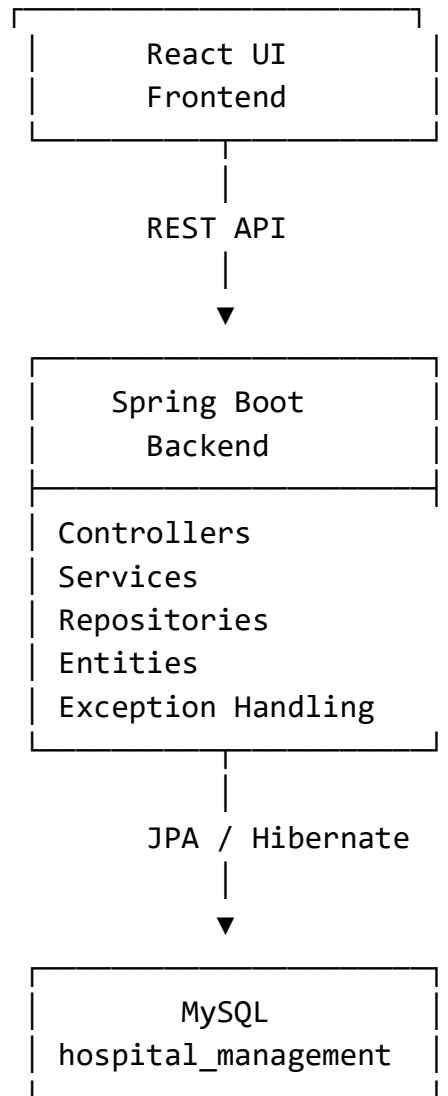
- Manage patients and doctors
- Manage appointments
- Monitor overall hospital statistics
- View appointment status information

# 4. Technology Stack

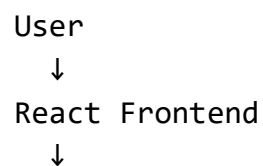
| Layer                | Technology                     |
|----------------------|--------------------------------|
| Frontend             | React.js                       |
| Programming Language | JavaScript                     |
| Backend              | Spring Boot                    |
| Backend Language     | Java                           |
| REST API             | Spring Web                     |
| Database             | MySQL                          |
| ORM                  | Spring Data JPA /<br>Hibernate |
| Build Tool           | Maven                          |
| Testing              | JUnit, Mockito                 |
| API Testing          | cURL                           |
| Version Control      | Git                            |
| Repository           | GitHub                         |
| IDE                  | IntelliJ IDEA / VS Code        |
| Database Tool        | MySQL Workbench                |

## 5. System Architecture

The application follows a three-layer architecture:



### Data Flow



REST API  
↓  
Spring Boot Controller  
↓  
Service Layer  
↓  
Repository Layer  
↓  
MySQL Database  
↓  
Response  
↓  
React Frontend

## 6. Project Structure

```
hospital-management-system/  
├── backend/  
│   ├── src/  
│   │   ├── main/  
│   │   │   ├── java/com/hospital/management/  
│   │   │   │   ├── controller/  
│   │   │   │   ├── entity/  
│   │   │   │   ├── exception/  
│   │   │   │   ├── repository/  
│   │   │   │   └── service/  
│   │   ├── test/  
│   │   └── resources/  
│   └── pom.xml  
└── frontend/  
    ├── src/  
    │   ├── components/  
    │   └── pages/
```

```
|
|
|   |   |   |
|   |   |   |— services/
|   |   |   |— App.jsx
|   |   |   |— App.css
|   |   |   |— index.css
|   |
|   |— package.json
|   |— index.html
|
|— docs/
|   |— requirements.md
|   |— database-design.md
|
|— .github/
|   |— copilot-instructions.md
|
|— .gitignore
|— README.md
```

## 7. Core Modules

### 7.1 Patient Management

The Patient module provides complete CRUD functionality.

#### Features

- Create patient
- View all patients
- View individual patient
- Update patient
- Delete patient
- Search patients
- View appointment history

#### Patient Information

The system stores:

- Patient Code
- First Name
- Last Name
- Date of Birth
- Gender
- Phone
- Email
- Blood Group
- Address
- Created/Updated timestamps

## Search

Patients can be searched using information such as:

- Patient code
- First name
- Last name
- Phone number

# 8. Doctor Management

The Doctor module manages doctor information.

## Features

- Create doctor
- View doctors
- View individual doctor
- Update doctor
- Delete doctor
- Filter by specialization
- View appointment schedule

## Doctor Information

The system stores:

- Doctor Code
- Doctor Name
- Specialization
- Phone
- Email
- Availability
- Created/Updated timestamps

## 9. Appointment Management

The Appointment module manages appointments between patients and doctors.

### Features

- Create appointment
- View appointments
- View individual appointment
- Update appointment
- Delete appointment
- Cancel appointment
- Filter appointments
- Detect appointment conflicts

### Appointment Information

The system stores:

- Appointment Code
- Patient
- Doctor
- Appointment Date
- Appointment Time
- Reason
- Status
- Notes
- Created/Updated timestamps

## 10. Appointment Conflict Detection

One of the important business rules implemented in the system is appointment conflict detection.

When creating an appointment, the system checks whether the selected doctor already has an appointment at the same:

Doctor + Date + Time

If a conflicting appointment already exists, the system rejects the new appointment.

For example:

Doctor: D001

Date: 2026-09-21

Time: 10:00

If another appointment already exists for the same doctor at that date and time, the new appointment cannot be created.

The same validation is also applied when updating an existing appointment.

## 11. Appointment Cancellation

Appointments can be cancelled without permanently removing the appointment record.

When the Cancel operation is performed:

Status

Scheduled

↓

Cancelled

The appointment remains stored in the database with:

status = Cancelled

This allows the system to retain the appointment record for historical purposes.

## 12. Patient Appointment History

Each patient has a **History** option.

The history displays the patient's previous and current appointments, including:

- Appointment Code
- Doctor
- Date
- Time
- Reason
- Status

The appointments are displayed in descending date/time order.

API:

GET /api/patients/{id}/appointments

## 13. Doctor Appointment Schedule

Each doctor has a **Schedule** option.

The schedule displays appointments associated with the selected doctor.

Information displayed includes:

- Appointment Code
- Patient
- Date
- Time
- Reason
- Status

Appointments are ordered chronologically.

API:

GET /api/appointments/doctor/{doctorId}

## 14. Dashboard

The dashboard provides an overview of the hospital data.

### Main Statistics

The dashboard displays:

- Total Patients
- Total Doctors
- Total Appointments

### Appointment Status

The dashboard also displays appointment counts by status:

- Scheduled
- Completed
- Cancelled

### Upcoming Appointments

The dashboard displays upcoming appointments while excluding cancelled appointments.

This provides a quick overview of the current appointment schedule.

## 15. Validation

Validation is implemented on the backend using Spring validation.

### Patient Validation

- Required fields must be provided.
- Email must have a valid format.

- Phone number must contain a valid number of digits.
- Date of birth cannot be a future date.
- Patient code must be unique.

## Doctor Validation

- Required fields must be provided.
- Doctor code must be unique.
- Email must follow a valid format.

## Appointment Validation

- Required appointment information must be provided.
- Patient must exist.
- Doctor must exist.
- Appointment code must be unique.
- Doctor cannot have conflicting appointments.

# 16. Exception Handling

The application uses centralized exception handling.

The following exceptions are handled:

DuplicatePatientCodeException  
 DuplicateDoctorCodeException  
 DuplicateAppointmentCodeException  
 PatientNotFoundException  
 DoctorNotFoundException  
 AppointmentNotFoundException  
 AppointmentConflictException

The `GlobalExceptionHandler` converts these exceptions into appropriate HTTP responses.

## HTTP Status Codes

| Situation          | HTTP Status |
|--------------------|-------------|
| Successful request | 200         |

|                                  |     |
|----------------------------------|-----|
| Resource created                 | 201 |
| Invalid request/validation error | 400 |
| Resource not found               | 404 |
| Duplicate/conflicting data       | 409 |

## 17. REST API Endpoints

### Patient APIs

| Method | Endpoint                        | Description                 |
|--------|---------------------------------|-----------------------------|
| POST   | /api/patients                   | Create patient              |
| GET    | /api/patients                   | Get all/search patients     |
| GET    | /api/patients/{id}              | Get patient                 |
| PUT    | /api/patients/{id}              | Update patient              |
| DELETE | /api/patients/{id}              | Delete patient              |
| GET    | /api/patients/{id}/appointments | Patient appointment history |

### Doctor APIs

| Method | Endpoint          | Description            |
|--------|-------------------|------------------------|
| POST   | /api/doctors      | Create doctor          |
| GET    | /api/doctors      | Get all/filter doctors |
| GET    | /api/doctors/{id} | Get doctor             |
| PUT    | /api/doctors/{id} | Update doctor          |
| DELETE | /api/doctors/{id} | Delete doctor          |

### Appointment APIs

| Method | Endpoint          | Description             |
|--------|-------------------|-------------------------|
| POST   | /api/appointments | Create appointment      |
| GET    | /api/appointments | Get/filter appointments |

|        |                                     |                    |
|--------|-------------------------------------|--------------------|
| GET    | /api/appointments/{id}              | Get appointment    |
| GET    | /api/appointments/doctor/{doctorId} | Doctor schedule    |
| PUT    | /api/appointments/{id}              | Update appointment |
| PUT    | /api/appointments/{id}/cancel       | Cancel appointment |
| DELETE | /api/appointments/{id}              | Delete appointment |

## Dashboard APIs

| Method | Endpoint                                 | Description                           |
|--------|------------------------------------------|---------------------------------------|
| GET    | /api/dashboard/counts                    | Get patient/doctor/appointment counts |
| GET    | /api/dashboard/appointment-status-counts | Get status counts                     |
| GET    | /api/dashboard/upcoming-appointments     | Get upcoming appointments             |

# 18. Database Design

The application uses MySQL database:

hospital\_management

The main entities are:

Patient

Doctor

Appointment

Relationship:

Patient 1 ————— \* Appointment \* ————— 1 Doctor

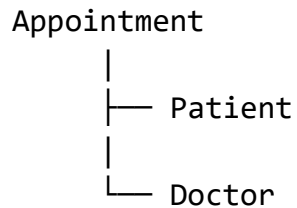
This means:

- One patient can have multiple appointments.
- One doctor can have multiple appointments.
- Each appointment belongs to one patient.
- Each appointment belongs to one doctor.

## 19. JPA Relationships

The Appointment entity uses relationships with Patient and Doctor.

Conceptually:



This is implemented using JPA `@ManyToOne` relationships.

Hibernate manages the database relationship and persistence.

## 20. Frontend Services

The React frontend communicates with the Spring Boot backend through service files.

### Patient Service

`patientService.js`

Responsible for patient API operations.

### Doctor Service

`doctorService.js`

Responsible for doctor API operations.

### Appointment Service

`appointmentService.js`

Responsible for appointment operations including:

- Fetch
- Create
- Update
- Cancel
- Delete
- Filtering

Additional services:

`patientAppointmentService.js`  
`doctorAppointmentService.js`

These handle patient history and doctor schedule requests.

## 21. Testing

The backend was tested using JUnit and Mockito.

Service-level test coverage includes:

### **PatientServiceTest**

8 tests

### **DoctorServiceTest**

8 tests

### **AppointmentServiceTest**

11 tests

### **Application Context Test**

1 test

## Total

28 tests  
0 failures  
0 errors  
0 skipped

The Maven test execution completed successfully with:

BUILD SUCCESS

## 22. Manual API Testing

REST APIs were manually tested using `curl`.

Examples include:

```
curl http://localhost:8080/api/patients
curl http://localhost:8080/api/doctors
curl http://localhost:8080/api/appointments
```

Appointment cancellation was tested using:

```
curl -X PUT http://localhost:8080/api/appointments/{id}/cancel
```

The response confirmed that the appointment status was changed to:

Cancelled

## 23. Running the Backend

Navigate to the backend directory:

```
cd backend
```

Set the database password environment variable:

```
export DB_PASSWORD='HospitalApp@123'
```

Run Maven:

```
mvn spring-boot:run
```

The backend starts at:

<http://localhost:8080>

## 24. Running the Frontend

Navigate to the frontend directory:

```
cd frontend
```

Install dependencies:

```
npm install
```

Start the development server:

```
npm run dev
```

The frontend is available at:

<http://localhost:5173>

## 25. Build Process

The backend uses Maven for build management.

Common Maven commands:

```
mvn clean
```

Removes previous build artifacts.

```
mvn compile
```

Compiles the Java source code.

```
mvn test
```

Runs the automated tests.

```
mvn package
```

Creates the packaged application artifact.

The Maven lifecycle can be used to clean, compile, test, package, and deploy the application.

## 26. Error and Success Feedback

The frontend provides user feedback for major operations.

Examples:

```
Patient created successfully.
```

```
Patient updated successfully.
```

```
Patient deleted successfully.
```

```
Doctor created successfully.
```

```
Doctor updated successfully.
```

```
Doctor deleted successfully.
```

```
Appointment created successfully.
```

```
Appointment updated successfully.
```

```
Appointment cancelled successfully.
```

```
Appointment deleted successfully.
```

Error responses from the backend are displayed to the user when an operation fails.

## 27. Git and Version Control

Git is used for source-code version control.

The project repository contains separate frontend and backend directories.

Development changes were committed incrementally, including:

- Backend implementation
- Frontend implementation
- Dashboard
- Patient appointment history
- Doctor appointment schedule
- Appointment cancellation
- Dashboard appointment status counts
- Patient UI feedback
- Doctor UI feedback
- Appointment UI feedback

The project is maintained on GitHub using the `main` branch.

## 28. Key Features / Innovations

The project includes several features beyond basic CRUD:

### 1. Appointment Conflict Detection

Prevents a doctor from being assigned two appointments at the same date and time.

### 2. Patient Appointment History

Allows staff to view the complete appointment history associated with a patient.

### 3. Doctor Appointment Schedule

Allows staff to view the appointments assigned to a particular doctor in chronological order.

### 4. Appointment Cancellation

Allows an appointment to be cancelled while retaining its database record.

### 5. Dashboard

Provides an overview of:

- Patients
- Doctors
- Appointments
- Appointment status
- Upcoming appointments

## 29. Future Enhancements

Possible future improvements include:

- JWT-based authentication
- Role-based access control
- Admin and staff-specific permissions
- Email/SMS appointment notifications
- Advanced dashboard charts
- Pagination
- Advanced appointment search
- Prescription management
- Billing management
- Medical record management
- Cloud deployment
- Automated CI/CD pipeline

## 30. Conclusion

The Hospital Management System provides a full-stack solution for managing essential hospital operations.

The system integrates a React frontend with a Spring Boot REST backend and MySQL database. It implements CRUD operations, validation, exception handling, appointment conflict detection, appointment cancellation, patient history, doctor schedules, and dashboard statistics.

The application was tested using automated JUnit/Mockito tests and manual REST API testing. The backend test suite successfully completed with **28 tests passing with zero failures and zero errors**.

